Java Coding Exercises

Object Casting

Q1.
Create a base class `Employee` with a method `work()`. Create a subclass `Manager` with an extra method `conductMeeting()`. Do upcasting to call `work()`, then downcasting to call `conductMeeting()`.

Q2.
Write a program with `class Animal` and subclass `Dog`. Store a `Dog` object inside an `Animal` reference (upcasting). Then safely downcast it back to `Dog` to call `bark()`.

Static Keyword

Q3.
Write a class `Student` with a static variable `count` that increases each time a new student object is created. Add a static method `displayCount()` to print the number of students created. Create 3 students in `main()` and display the total count.

Q4.
Write a program where a `BankAccount` class has a static variable `interestRate`, a method to calculate interest using it. Change the `interestRate` in `main()` and see the effect on multiple accounts.

Super Keyword

Q5.
Write two classes: `Vehicle` (constructor prints "Vehicle created") and `Car` (constructor calls `super()` and prints "Car created"). Create a `Car` object in `main()`.

Q6.
Write a parent class `Person` with a method `displayInfo()`. Write a subclass `Student` that has a method `showInfo()` which calls the parent method using `super.displayInfo()`.

Inheritance

Q7.
Create a base class `Shape` with a method `area()`. Derive two classes `Rectangle` and `Circle` that override the `area()` method. In `main()`, calculate and print the area of both.

Q8.
Implement multilevel inheritance: `LivingThing` (method `breathe()`), `Animal` extends `LivingThing` (method `eat()`), `Dog` extends `Animal` (method `bark()`). In `main()`, create a `Dog` object and call all three methods.

Q9.
Implement hierarchical inheritance: `Vehicle` (method `start()`), `Car` and `Bike` extend `Vehicle` and override `start()`. In `main()`, create objects of `Car` and `Bike` and call their `start()` methods.

Polymorphism

Q10.

Create a class `Employee` with method `calculateSalary()`. Subclass `FullTimeEmployee` and `PartTimeEmployee` override the method. In `main()`, use an `Employee` reference to call the method on both objects.

Q11.

Write a program to demonstrate method overloading in a class `Calculator`: `add(int a, int b)`, `add(double a, double b)`, `add(int a, int b, int c)`. Test all versions in `main()`.

Q12.

Write a program with base class `Shape` having a method `draw()`. Subclasses `Circle`, `Square`, and `Triangle` override `draw()`. Store all objects in an array of `Shape` references. Use a loop to call `draw()` on each.